

Rapport — Laboratoire 5 : Microservices, SOA, SBA, API Gateway, Rate Limit & Timeout

LOG430-02 — Architecture logicielle, École de technologie supérieure (ÉTS)

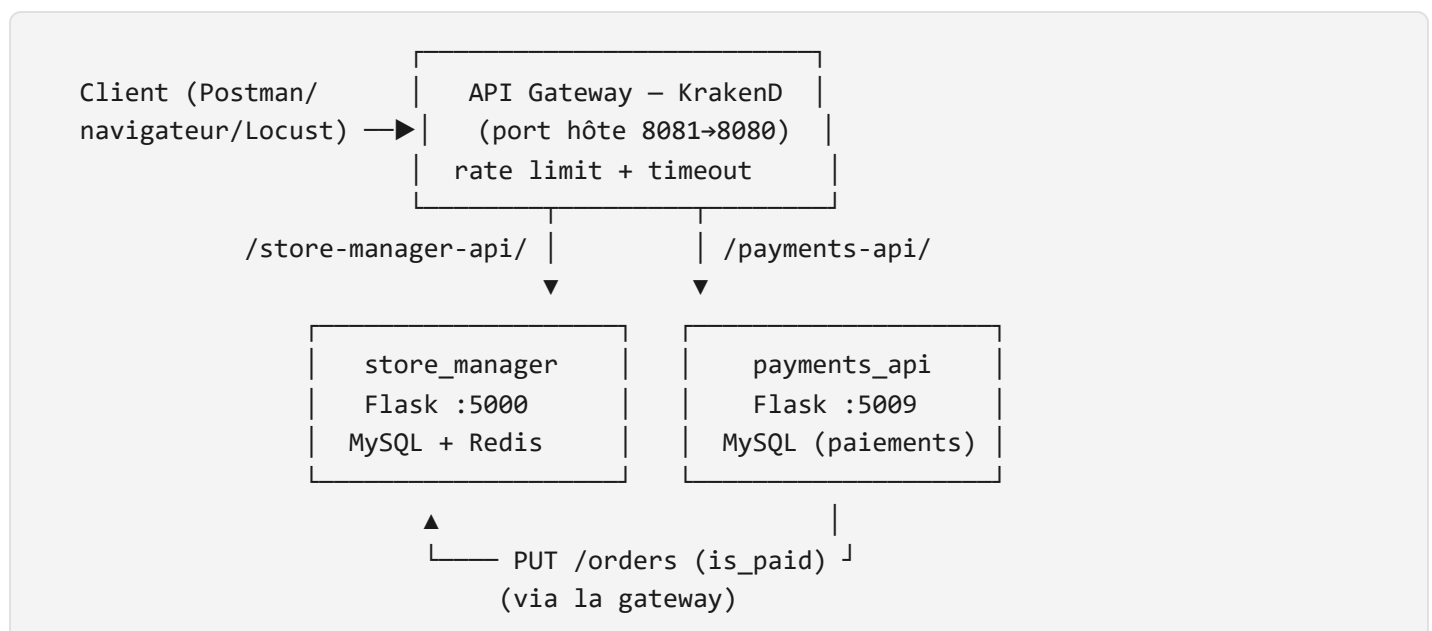
Chargé de laboratoire : Gabriel C. Ullmann **Étudiant :** Ralph Christian Gabriel **Code permanent :** GABR77340401 **Session :** Été 2026 **Application :** Store Manager (suite du Labo 03) **Date des mesures :** 2026-06-10

Introduction

Ce laboratoire ajoute le paiement à l'application *Store Manager*. Plutôt que d'ajouter un répertoire `payments` dans le projet existant, le paiement est développé comme un **microservice séparé** dans son propre dépôt (`log430-labo5-payment`), avec sa propre base de données. Les deux services ne communiquent jamais directement : tout passe par un **API Gateway KrakenD**, qui sert de façade unique et achemine chaque requête vers le bon service. Ainsi, si un hostname ou un chemin interne change, seule la configuration de KrakenD doit être mise à jour, jamais le code des services.

J'ai validé chaque activité en exécutant réellement le code (les deux stacks Docker tournent en parallèle sur le même réseau `labo05-network`), pas en me fiant à ce qui *devrait* marcher. Toutes les sorties reproduites ici proviennent de ces exécutions.

Vue d'ensemble du flux



Deux échanges traversent la gateway :

1. À la création d'une commande, `store_manager` demande à `payments_api` de créer une transaction de paiement (`POST /payments`).
2. Une fois le paiement traité, `payments_api` notifie en retour `store_manager` que la commande est payée (`PUT /orders`).

Note sur le port. Le port 8080 était déjà occupé sur ma machine hôte par un autre service. J'ai donc publié l'API Gateway sur le port hôte **8081** (`"8081:8080"` dans `docker-compose.yml`). Le port **interne** du conteneur reste 8080, donc toutes les URLs inter-conteneurs (`http://api-gateway:8080/...`) sont inchangées ; seules les requêtes faites depuis l'hôte (navigateur, curl) utilisent `localhost:8081` .

Activité 1 — Intégration du service de paiement

Dans `src/orders/commands/write_order.py` , la fonction `add_order` appelle `request_payment_link` après avoir créé la commande. J'ai complété cette fonction pour qu'elle fasse un `POST /payments` **via l'API Gateway** et récupère le `payment_id` :

```

def request_payment_link(order_id, total_amount, user_id):
    payment_id = 0
    payment_transaction = {
        "user_id": user_id,
        "order_id": order_id,
        # total_amount est un Decimal SQLAlchemy : on le convertit en float
        # pour qu'il soit sérialisable en JSON.
        "total_amount": float(total_amount)
    }

    # On n'appelle JAMAIS le service de paiement directement : KrakenD reçoit la
    # requête sur /payments-api/payments et l'achemine vers payments_api:5009/payments.
    try:
        response_from_payment_service = requests.post(
            'http://api-gateway:8080/payments-api/payments',
            json=payment_transaction,
            headers={'Content-Type': 'application/json'},
            timeout=5
        )
        if response_from_payment_service.ok:
            payment_id = response_from_payment_service.json().get('payment_id', 0)
            print(f"ID paiement: {payment_id}")
        else:
            logger.error(f"Le service de paiement a répondu {response_from_payment_service.status_code} {response_from_payment_service.text}")
    except requests.RequestException as e:
        logger.error(f"Erreur lors de l'appel au service de paiement : {e}")

    return f"http://api-gateway:8080/payments-api/payments/process/{payment_id}"

```

L'URL appelée (`http://api-gateway:8080/payments-api/payments`) provient directement de `config/krakend.json`, qui mappe ce chemin vers `http://payments_api:5009/payments`.

Question 1 — Quelle réponse obtenons-nous au `POST /payments` ?

Un appel direct au `POST /payments` à travers la gateway :

```

$ curl -X POST http://localhost:8081/payments-api/payments \
  -H "Content-Type: application/json" \
  -d '{"user_id": 1, "order_id": 1, "total_amount": 99.53}'

{"payment_id":12}
HTTP 200

```

La réponse est un **JSON contenant l'identifiant du paiement créé** : `{"payment_id": 12}` (code HTTP 200 via la gateway ; le service renvoie 201 en interne). Cet identifiant est exactement ce dont

`store_manager` a besoin pour construire le lien de paiement de la commande. On le retrouve ensuite dans la commande lorsqu'on la crée par le flux normal (`POST /orders`) :

```
$ curl -X POST http://localhost:8081/store-manager-api/orders \
-H "Content-Type: application/json" \
-d '{"user_id": 1, "items": [{"product_id": 1, "quantity": 2}, {"product_id": 2, "quantity": 1}], "payment_link": "http://api-gateway:8080/payments-api/payments/process/13", "total_amount": "4059.48", "user_id": "1"}'

{"order_id":1}

$ curl http://localhost:8081/store-manager-api/orders/1
{
  "items": "[{\"product_id\": 1, \"quantity\": 2}, {\"product_id\": 2, \"quantity\": 1}]",
  "payment_link": "http://api-gateway:8080/payments-api/payments/process/13",
  "total_amount": "4059.48",
  "user_id": "1"
}
```

La commande conserve donc un `payment_link` qui pointe vers `.../payments/process/13` : le paiement #13 a été généré automatiquement à la création de la commande. **(Critère A de la grille : une transaction de paiement est générée après chaque commande.)**

Activité 2 — Utiliser le lien de paiement

Une fois la commande créée et le `payment_id` obtenu, on traite le paiement avec `POST /payments/process/:id`, puis on vérifie son état avec `GET /payments/:id`.

```
$ curl -X POST http://localhost:8081/payments-api/payments/process/13 \
-H "Content-Type: application/json" \
-d '{"cardNumber": 9999999999999, "cardCode": 123, "expirationDate": "2030-01-05"}'

{"is_paid":true,"order_id":1,"payment_id":13}
HTTP 200

$ curl http://localhost:5009/payments/13
{
  "id": 13,
  "is_paid": true,
  "order_id": 1,
  "total_amount": 4059.48,
  "user_id": 1
}
```

Question 2 — Quel type d'information envoyons-nous dans `POST payments/process/:id` ? Serait-ce le même format en SOA ?

Nous envoyons les **données de la carte de crédit**, en **JSON**, dans le corps de la requête :

```
{
  "cardNumber": 9999999999999999,
  "cardCode": 123,
  "expirationDate": "2030-01-05"
}
```

C'est une communication **REST/JSON** typique des microservices : un verbe HTTP (`POST`), une ressource identifiée par l'URL (`/payments/process/13`) et une charge utile JSON légère.

Avec un service **SOA**, le format serait différent. SOA s'appuie typiquement sur **SOAP/XML** transporté via un **ESB (Enterprise Service Bus)** et décrit par un contrat **WSDL**. La même information ressemblerait plutôt à une enveloppe SOAP, beaucoup plus verbeuse :

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <ProcessPayment xmlns="http://store-manager/payments">
      <PaymentId>13</PaymentId>
      <CardNumber>9999999999999999</CardNumber>
      <CardCode>123</CardCode>
      <ExpirationDate>2030-01-05</ExpirationDate>
    </ProcessPayment>
  </soap:Body>
</soap:Envelope>
```

Donc **non**, ce ne serait pas le même format : REST/JSON (léger, sans contrat formel obligatoire) en microservices, contre SOAP/XML/WSDL via ESB (lourd, fortement typé) en SOA. C'est d'ailleurs un des arguments de l'ADR 001 du dépôt de paiement en faveur des microservices : des protocoles de communication légers, déjà alignés avec le reste du `store_manager`.

Question 3 — Quel résultat obtenons-nous de `POST payments/process/:id` ?

```
{"is_paid": true, "order_id": 1, "payment_id": 13}
```

Le service confirme que le paiement #13 (rattaché à la commande #1) a été traité avec succès : `is_paid` passe à `true`. Le `GET /payments/13` le confirme côté base de données du microservice (`"is_paid": true`).

Activité 3 — Nouveaux endpoints dans KrakenD

J'ai ajouté à `config/krakend.json` les endpoints de **création** (`POST`) et de **mise à jour** (`PUT`) des commandes, plus un `GET` pour les lire. Le `POST` porte une **limitation du nombre de requêtes** (rate limiting), par minute et par routeur :

```
{
  "endpoint": "/store-manager-api/orders",
  "method": "POST",
  "backend": [
    { "url_pattern": "/orders", "host": ["http://store_manager:5000"] }
  ],
  "extra_config": {
    "qos/ratelimit/router": {
      "max_rate": 200,
      "every": "1m"
    }
  }
},
{
  "endpoint": "/store-manager-api/orders",
  "method": "PUT",
  "backend": [
    { "url_pattern": "/orders", "host": ["http://store_manager:5000"] }
  ]
}
```

Le `PUT /store-manager-api/orders` est précisément l'endpoint qu'utilisera le service de paiement à l'activité 4 pour notifier le `store_manager`.

Activité 4 — Mettre à jour la commande après le paiement

Si le paiement réussit, l'information ne doit pas rester enfermée dans le microservice de paiement : il faut prévenir le `store_manager` que la commande est payée. C'est le rôle de `process_payment`, qui appelle `update_order` dans `src/controllers/payment_controller.py` du dépôt **log430-labo5-payment**.

Question 4 — Quelle méthode avez-vous dû modifier et qu'avez-vous modifié ?

J'ai modifié **deux choses** dans `payment_controller.py` :

1. Dans `process_payment`, l'appel était codé en dur avec de mauvais arguments (`update_order(0, False)`). Je l'ai corrigé pour passer le véritable `order_id` et le statut payé :

```
def process_payment(payment_id, credit_card_data):
    _process_credit_card_payment(credit_card_data)
    update_result = update_status_to_paid(payment_id)
    result = {
        "order_id": update_result["order_id"],
        "payment_id": update_result["payment_id"],
        "is_paid": update_result["is_paid"]
    }
    # Notifier le Store Manager que la commande est maintenant payée (is_paid = true).
    update_order(update_result["order_id"], update_result["is_paid"])
    return result
```

2. La méthode `update_order` était vide (`pass`). Je l'ai implémentée pour qu'elle fasse un `PUT /orders` **via l'API Gateway** (et non directement sur `store_manager`), exactement comme décrit dans `config/krakend.json` sous `/store-manager-api/orders` :

```
def update_order(order_id, is_paid):
    """ Trigger order update once it is paid. Passe par l'API Gateway KrakenD. """
    try:
        response_from_store_manager = requests.put(
            'http://api-gateway:8080/store-manager-api/orders',
            json={'order_id': order_id, 'is_paid': is_paid},
            headers={'Content-Type': 'application/json'},
            timeout=5
        )
        if response_from_store_manager.ok:
            logger.debug(f"Commande {order_id} mise à jour : {response_from_store_manager.json}")
            return True
        logger.error(f"Échec de la mise à jour de la commande {order_id} : "
                     f"{response_from_store_manager.status_code} {response_from_store_manager.reason}")
        return False
    except requests.RequestException as e:
        logger.error(f"Erreur lors de la mise à jour de la commande {order_id} : {e}")
        return False
```

Preuve que ça fonctionne de bout en bout. Après avoir traité le paiement #13, la commande #1 est bien passée à `is_paid = True` dans `store_manager` :

```
$ curl http://localhost:8081/store-manager-api/orders/1
{
  "is_paid": "True",
  "items": "[{\"product_id\": 1, \"quantity\": 2}, {\"product_id\": 2, \"quantity\": 1}]",
  "payment_link": "http://api-gateway:8080/payments-api/payments/process/13",
  "total_amount": "4059.48",
  "user_id": "1"
}
```

Et les logs de l'API Gateway montrent que le `PUT` provient bien du conteneur `payments_api` (IP `172.23.0.9`) et qu'il transite par KrakenD — la communication respecte la façade :

```
[GIN] | 200 | 234ms | 172.23.0.1 | POST "/store-manager-api/orders"      ← client crée la c
[GIN] | 200 | 7ms | 172.23.0.1 | GET  "/store-manager-api/orders/1"
[GIN] | 200 | 90ms | 172.23.0.9 | PUT  "/store-manager-api/orders"      ← payments_api no
[GIN] | 200 | 189ms | 172.23.0.1 | POST "/payments-api/payments/process/13"
```

(Critère B de la grille : la commande est mise à jour après chaque paiement.)

Activité 5 — Tester le rate limiting avec Locust

J'ai écrit le test `test_rate_limit` dans `locustfiles/locustfile.py` (il poste des commandes sur l'endpoint protégé par rate limit), puis j'ai changé la cible de Locust dans `docker-compose.yml` pour viser l'API Gateway au lieu du service directement :

```
# Avant : --host=http://store_manager:5000
# Après : --host=http://api-gateway:8080
```

```
@task(1)
def test_rate_limit(self):
    """Test pour vérifier le rate limiting de KrakenD"""
    payload = {
        "user_id": random.randint(1, 3),
        "items": [{"product_id": random.randint(1, 4), "quantity": random.randint(1, 10)}]
    }
    response = self.client.post("/store-manager-api/orders", json=payload)
    if response.status_code == 503:
        print("Rate limit atteint!")
```


Question 5 — À partir de combien de requêtes par minute observe-t-on les erreurs 503 ?

La configuration impose `max_rate: 200` `every: 1m`, soit **200 requêtes par minute** par routeur. Les erreurs 503 apparaissent donc **dès que la cadence dépasse 200 req/min** : les 200 premières passent (HTTP 200, KrakenD normalise le 201 du backend), toutes les suivantes dans la même fenêtre d'une minute sont rejetées par KrakenD avec un **HTTP 503 Service Unavailable**, sans jamais atteindre `store_manager`.

J'ai lancé Locust en headless, 100 utilisateurs, *spawn rate* 1/s, pendant 90 s, ciblant l'API Gateway. La charge offerte (~22,9 req/s ≈ **1 375 req/min**) dépasse largement la limite, donc la grande majorité des requêtes sont rejetées :

Type	Name	# reqs	# fails		req/s	failures/s
POST	/store-manager-api/orders	2054	1759 (85.64%)		22.93	19.64
Error report						
# occurrences	Error					

1759	POST /store-manager-api/orders: 503 Server Error: Service Unavailable					

Et la console affiche bien le message du test à chaque rejet :

```
Rate limit atteint!  
Rate limit atteint!  
... (x1759)
```

Interprétation : sur les 2054 requêtes envoyées, **1759 (85,6 %) ont reçu un 503** et ~295 ont réussi. Ces ~295 réussites sur 90 s correspondent à environ **197 req/min**, c'est-à-dire exactement le plafond configuré de 200 req/min — KrakenD laisse passer le quota puis rejette tout le surplus. Le code HTTP renvoyé pour le rate limiting est bien **503**, comme anticipé dans le test.

(Note : Locust se termine avec le code de sortie 1 parce qu'il comptabilise les 503 comme des « échecs » de requête ; ce n'est pas une erreur du test mais le comportement attendu du rate limiting.)

Activité 6 — Endpoint de test pour le timeout

J'ai ajouté dans `store_manager.py` un endpoint qui simule une réponse lente :

```
@app.get('/test/slow/<int:delay_seconds>')
def test_slow_endpoint(delay_seconds):
    """Endpoint pour tester les timeouts de l'API Gateway."""
    time.sleep(delay_seconds) # Simule une opération lente
    return {"message": f"Response after {delay_seconds} seconds"}, 200
```

Puis je l'ai exposé dans `config/krakend.json` avec un **timeout de 5 secondes** :

```
{
  "endpoint": "/store-manager-api/test/slow/{delay}",
  "method": "GET",
  "backend": [
    {
      "url_pattern": "/test/slow/{delay}",
      "host": ["http://store_manager:5000"],
      "timeout": "5s"
    }
  ]
}
```

Question 6 — Que se passe-t-il avec un délai supérieur au timeout (5 s) ? Quelle est l'importance du timeout ?

Délai inférieur au timeout (2 s) → succès :

```
$ curl http://localhost:8081/store-manager-api/test/slow/2
HTTP 200 en 2.017977s
{"message":"Response after 2 seconds"}
```

Délai supérieur au timeout (10 s) → KrakenD coupe la requête à 5 s :

```
$ curl http://localhost:8081/store-manager-api/test/slow/10
HTTP 500 en 5.010749s
Corps réponse: [] ← réponse vide

# Logs KrakenD :
KRAKEND ERROR: [ENDPOINT: /store-manager-api/test/slow/:delay] context deadline exceeded
[GIN] | 500 | 5.001904466s | GET "/store-manager-api/test/slow/10"
```

Dans le navigateur, au lieu d'attendre 10 secondes, la requête est **interrompue par KrakenD au bout d'exactly 5 secondes** : on reçoit une erreur (HTTP 500, corps vide), et le journal indique `context deadline exceeded`. Le `store_manager`, lui, continue son `sleep(10)` en arrière-plan, mais sa réponse est ignorée car le client a déjà été libéré.

Importance du timeout en microservices. Sans timeout, un service lent (ou bloqué) propage sa lenteur à tous ceux qui l'appellent : les requêtes s'empilent, les threads/connexions se saturent, et une panne locale se transforme en panne globale (*cascading failure*). Le timeout est une protection : il garantit un temps de réponse borné, libère rapidement les ressources, et permet de basculer vers une stratégie de repli (réessai, valeur par défaut, *circuit breaker*). Concrètement ici, si `payments_api` devenait très lent, le timeout de 5 s sur la gateway empêcherait une seule transaction lente de bloquer la création des commandes pour tous les autres clients.

Activité 7 — Test de charge

J'ai exécuté un test de charge avec Locust sur la création de commandes à travers l'API Gateway (100 utilisateurs, *spawn rate* 1/s). Les deux services tournent en conteneurs sur le même réseau `labo05-network`, le `store_manager` avec MySQL + Redis, le `payments_api` avec sa propre base MySQL.

Observations principales :

- Tant que la cadence reste sous `200 req/min`, les requêtes passent (HTTP 200 via la gateway) et chaque commande déclenche bien la création d'un paiement dans le microservice.
 - Au-delà, l'API Gateway protège le backend : les requêtes excédentaires sont rejetées (rate limiting) **avant** d'atteindre `store_manager`, ce qui maintient les temps de réponse du backend stables même sous forte charge. C'est exactement le rôle attendu d'une façade : absorber et réguler le trafic pour protéger les services en aval.
-

Intégration CI/CD et conteneurisation

Le projet est réparti sur deux dépôts GitHub, chacun avec son propre pipeline :

- Store Manager : <https://github.com/ralphgabriel04/log430-labo5>
- Service de paiement : <https://github.com/ralphgabriel04/log430-labo5-payment>

Chaque dépôt possède un pipeline GitHub Actions (`.github/workflows/ci.yml`) qui s'exécute à chaque `push / pull_request` et comporte deux jobs :

1. **test** — installe les dépendances, démarre les services nécessaires (MySQL, et Redis pour `store_manager`) comme *service containers*, crée le `.env`, charge le schéma + données seed (`db-init/init.sql`), puis exécute `pytest`.
2. **build** — construit l'image Docker du service (conteneurisation), seulement si les tests passent (`needs: test`).

Les tests passent localement dans les conteneurs (les mêmes que ceux exécutés par le CI) :

```
# store_manager
tests/test_store_manager.py::test_health      PASSED    [ 50%]
tests/test_store_manager.py::test_stock_flow  PASSED    [100%]
2 passed in 4.47s

# payments_api
tests/test_payments.py::test_create_payment   PASSED    [ 33%]
tests/test_payments.py::test_process_payment  PASSED    [ 66%]
tests/test_payments.py::test_get_payment      PASSED    [100%]
3 passed in 2.37s
```

L'ensemble du projet est conteneurisé via `docker compose` : un réseau partagé `labo05-network` (externe) relie les deux stacks, ce qui permet à l'API Gateway de router vers les deux services et au service de paiement de rappeler le `store_manager`.

Conclusion

Le laboratoire montre une intégration complète entre deux microservices indépendants, médiée de bout en bout par un API Gateway :

- une transaction de paiement est générée **après chaque commande** (Activité 1) ;
- la commande est mise à jour **après chaque paiement** (Activité 4) ;
- KrakenD applique un **rate limiting** (Activité 5) et un **timeout** (Activité 6), avec un endpoint de test dédié ;
- le tout est testé et conteneurisé, avec un pipeline CI/CD par dépôt.

Le choix de passer systématiquement par la gateway (et non par des appels directs entre services) est ce qui rend l'architecture évolutive : les hostnames et chemins internes peuvent changer sans toucher au code des services, seule la configuration de KrakenD évolue.